

67 Основы безопасности веб-приложений. XSS, CSRF, SQL Injection, безопасная обработка данных

Что такое безопасность веб-приложения

Безопасность — состояние защищённости личности, общества, государства и среды жизнедеятельности от внутренних и внешних угроз или опасностей.

«Security by design» (безопасность по замыслу) — это подход к разработке, при котором вопросы безопасности интегрируются в архитектуру и процесс создания продукта с самых ранних этапов, а не добавляются позже. Основная идея заключается в том, чтобы встраивать защиту с самого начала, чтобы минимизировать уязвимости и проактивно устранять риски. Этот подход требует, чтобы безопасность рассматривалась наравне с функциональными требованиями и влияла на все этапы жизненного цикла продукта.

В таблице ниже представлены главные принципы данного подхода.

Принцип	Краткое описание
Проактивная, а не реактивная защита	Предвидеть угрозы и встраивать защитные механизмы на этапе проектирования, а не после инцидента.
Минимизация поверхности атаки	Сокращать количество уязвимых точек (кода, сервисов, портов) до абсолютного минимума.
Принцип наименьших привилегий (PoLP)	Любой пользователь, процесс или система должны иметь доступ только к тем ресурсам, которые необходимы для выполнения их задач.
Глубина защиты (Defense in Depth)	Создание многослойной защиты, так что при компрометации одного слоя следующий продолжает обеспечивать безопасность.
Разделение обязанностей (Separation of Duties)	Критичные операции требуют участия нескольких субъектов для завершения, что предотвращает злоупотребления.
Простота и открытость дизайна	Безопасность не должна зависеть от неясности реализации. Архитектура и механизмы защиты должны быть простыми для анализа.
Недоверие по умолчанию (Zero Trust)	"Никогда не доверяй — всегда проверяй". Все запросы к системам и данным должны аутентифицироваться и авторизоваться, независимо от их источника.
Непрерывная проверка и мониторинг	Безопасность — это непрерывный процесс. Системы должны постоянно отслеживаться на предмет аномалий и проверяться.
Учет человеческого фактора	Дизайн системы должен быть интуитивным и предотвращать ошибки пользователей, ведущие к нарушению безопасности.
Безопасность по умолчанию (Secure by Default)	Продукт "из коробки" должен быть максимально безопасным, без необходимости дополнительной сложной настройки.
Фиксация и аудит событий	Все значимые события (особенно связанные с безопасностью) должны логироваться и защищаться от изменений для последующего анализа.

Если происходит взлом или хищение конфиденциальных данных пользователей ресурса, то владелец может понести серьезные последствия: финансовые (штрафы от регулирующих органов, судебные иски от клиентов, расходы на восстановление), репутационные (потеря клиентов) и даже уголовную ответственность за нарушение законодательства о персональных данных. Кроме того, возможно заражение сайта и его клиентов вредоносным ПО.

GDPR и ФЗ-152 — это законы о защите персональных данных, но с разными уровнями детализации и сферой применения. GDPR — это комплексный регламент ЕС с жесткими требованиями, а ФЗ-152 — это рамочный закон России, который задает общие принципы и опирается на подзаконные акты. Ключевые отличия включают: разграничение ролей (контроллер/процессор в GDPR), более строгие требования к согласию на обработку данных (в том числе для несовершеннолетних), право субъекта на перенос данных и более широкий список инцидентов, подлежащих уведомлению.

Ссылка на полный текст закона и все его статьи по данной гиперссылки.

Аспект	GDPR (Общий регламент по защите данных, ЕС)	ФЗ-152 "О персональных данных" (РФ)
Подход и уровень детализации	Комплексный, детально описывает все процессы обработки, права и обязанности. Имеет прямое действие в странах ЕС.	Рамочный, устанавливает общие требования и принципы. Детализация в подзаконных актах, указах и методических рекомендациях (в основном от Роскомнадзора).
Территориальное действие	Применяется ко всем организациям, обрабатывающим данные субъектов в ЕС, независимо от места нахождения организации.	В основном применяется к операторам, осуществляющим обработку на территории РФ.
Правовое основание для обработки	Шесть оснований (ст. 6): согласие, договор, законный интерес, выполнение правовой обязанности, защита жизненных интересов, общественный интерес. Согласие должно быть явным и информированным.	В основном акцент на предварительное согласие субъекта. Также есть иные основания, аналогичные GDPR (договор, исполнение закона и т.д.), но на практике согласие часто является основным.
Согласие для несовершеннолетних	Для детей до 16 лет (государства-члены могут снизить планку до 13 лет) требуется согласие законного представителя.	Четкого возрастного порога в самом законе нет. Требования уточняются в подзаконных актах. На практике для лиц до 18 лет часто требуется согласие родителей.
Основные роли при обработке	Четкое разделение: - Контроллер — определяет цели и средства обработки. - Процессор — обрабатывает данные по поручению контроллера. Ответственность распределена, но контроллер несет основную.	Один оператор, который определяет цели и средства обработки. Он может поручить обработку другому лицу, но несет полную ответственность перед субъектом и регулятором за действия такого лица.
Право на перенос данных (Data Portability)	Присутствует. Субъект имеет право получить свои данные в машиночитаемом формате и передать их другому контроллеру.	Отсутствует в явном виде. Субъект может запросить информацию об обработке и уточнить данные, но права на автоматический перенос между сервисами нет.
Право на информацию	Активная обязанность: информация предоставляется субъекту в момент сбора данных (прозрачность).	Реактивная обязанность: информация предоставляется по запросу субъекта.
Право на забвение	Закреплено. Субъект имеет право на удаление своих данных при определенных условиях.	Есть схожее право: субъект может отозвать согласие и потребовать прекратить обработку и уничтожить данные.
Оповещение об утечках	Обязательство уведомить надзорный орган в течение 72 часов, а субъектов данных — без неоправданной задержки, если утечка создает высокий риск для их прав и свобод.	Обязательство уведомить Роскомнадзор и ФСТЭК (ГосСОПКА) в установленные сроки (как правило, 24 часа с момента обнаружения). Уведомление субъектов данных не является обязательным, если приняты меры, делающие данные недоступными.
Назначение Ответственного за защиту данных (DPO)	Обязательно при определенных условиях (масштабная обработка, обработка спецкатегорий данных и т.д.).	Не обязателен по закону, но может быть назначен по инициативе оператора.
Штрафы и санкции	Очень высокие: до 20 млн евро или 4% от годового мирового оборота компании (что больше). Двухуровневая система штрафов.	Значительно ниже. Устанавливаются КоАП РФ (ст. 13.11) и зависят от типа нарушения. Максимальный штраф для юрлиц — до 1 млн рублей (или приостановление деятельности).
Ведение реестра обработки	Обязательство для компаний вести внутренний реестр операций обработки (ст. 30). Публикация не требуется.	Операторы обязаны уведомлять Роскомнадзор о начале обработки данных (с некоторыми исключениями). Данные вносятся в публичный реестр.

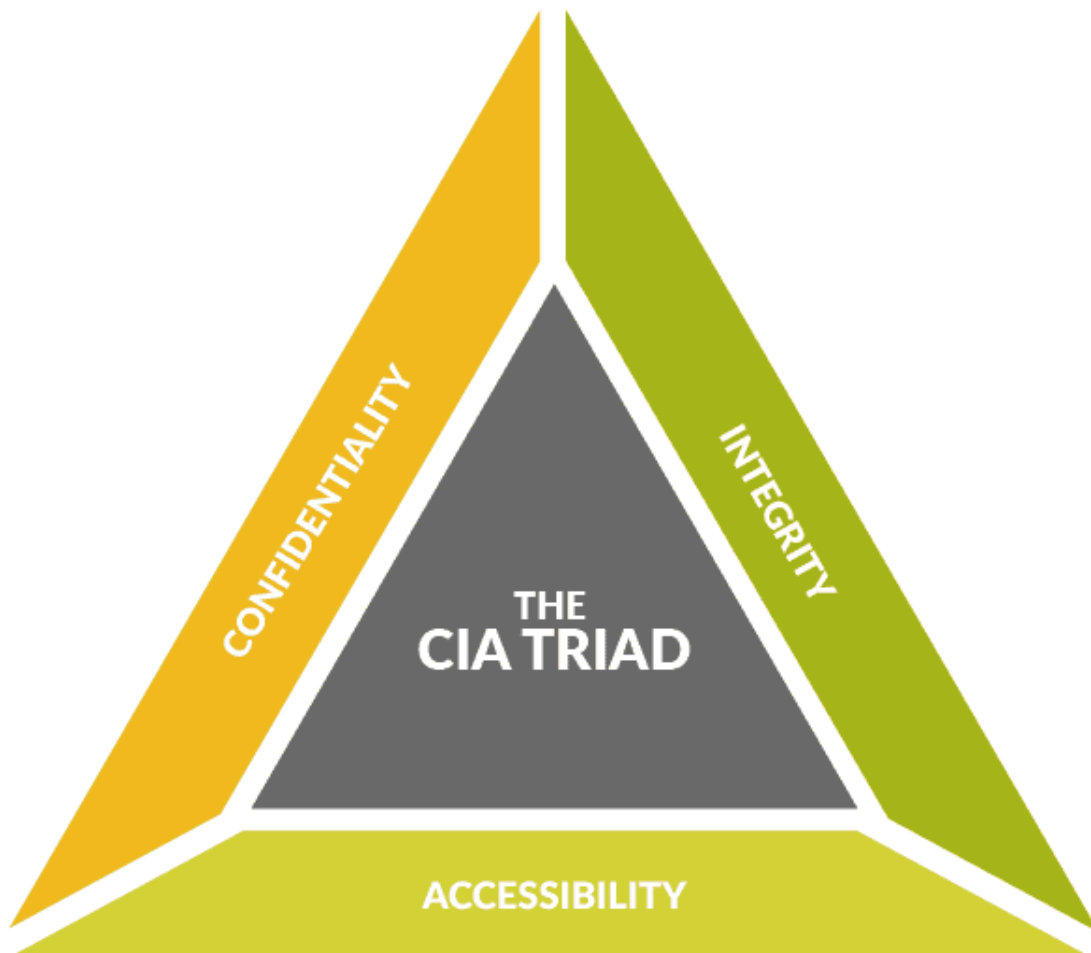
RED Security SOC — это центр круглосуточного мониторинга и реагирования на кибератаки компании RED Security, который 13 августа 2026 года был признан лидером по качеству и функциональности в исследовании «Квадрант Технологий».

За первые девять месяцев 2026 года эксперты зафиксировали и помогли отразить более 105 тысяч кибератак (из них почти 20 тысяч — высокой степени критичности). В первом полугодии 2026 года зафиксирован почти трехкратный рост атак на сектор здравоохранения, а в третьем квартале общий объем атак вырос на 73% по сравнению с предыдущими периодами. Основной удар (около 77%) приходится на объекты критической информационной инфраструктуры (КИИ) РФ.

Также, в данной статье от лаборатории Касперского рассказывается о ключевых атаках на российские отрасли.

Триада CIA (Confidentiality, Integrity, Accessibility)

Confidentiality, Integrity, Availability (CIA) — три основных составляющих информационной безопасности. Вместе они формируют триаду, которая определяет, как данные должны быть защищены.



- **Конфиденциальность (Confidentiality)**: гарантирует, что только авторизованные пользователи имеют доступ к информации. Это предотвращает несанкционированный доступ к конфиденциальным данным.
- **Целостность (Integrity)**: обеспечивает точность, полноту и надежность данных, защищая их от несанкционированного изменения.
- **Доступность (Availability)**: гарантирует, что авторизованные пользователи имеют доступ к информации и системам по мере необходимости. Это включает в себя поддержание работоспособности оборудования и инфраструктуры.

Never Trust User Input

Принцип «**Never trust user input**» («**никогда не доверяй вводу пользователя**») — это базовое правило безопасности в разработке программного обеспечения, согласно которому любые данные, поступающие извне (от пользователя, клиента, сторонних API или датчиков), потенциально опасны и требуют проверки.

Злоумышленник может обойти фронтенд-валидацию, отправить запрос через консоль, с помощью cURL или модифицировать трафик. Цель данного принципа — защитить систему от внедрения вредоносного кода, логических ошибок и несанкционированного доступа.

- **Злой умысел:** злоумышленники могут отправлять вредоносные данные, чтобы спровоцировать непреднамеренное поведение, такое как внедрение вредоносного кода или команд.
- **Неожиданные данные:** даже без злого умысла пользователи могут предоставлять данные в неожиданных форматах (например, неправильный тип данных, значение, выходящее за пределы допустимого диапазона), что может привести к сбою или неправильной работе приложения.
- **Обход проверок на стороне клиента:** полагаться исключительно на проверку на стороне клиента (то, что пользователь видит в своем браузере) небезопасно, поскольку пользователь может легко обойти эти проверки, манипулируя запросом.

Классификация угроз и место XSS\CSRF\SQLi

По критериям

По месту возникновения уязвимости

Уровень / Место возникновения	Категория угроз	Конкретные примеры	Краткое описание / Механизм
Угрозы на уровне сети	Перехват данных	Sniffing (прослушивание сети), MitM-атаки (Man-in-the-Middle)	Незаконный перехват и анализ сетевого трафика для кражи данных (логины, платежные данные).
	Сканирование и разведка	Порт-сканирование (Nmap), Сканирование уязвимостей (Nessus)	Сбор информации о топологии сети, открытых портах и версиях ПО для поиска векторов атаки.
	Подмена доверия (Spoofing)	IP-spoofing, DNS-spoofing / DNS cache poisoning	Подмена сетевых адресов для обмана систем аутентификации или перенаправления трафика.
	Атаки на отказ в обслуживании (DoS/DDoS)	Flood-атаки (SYN-flood, UDP-flood), Усилительные атаки (NTP Amplification, DNS Amplification)	Направление огромного объема ложного трафика на цель с целью истощения ее ресурсов.
	Компрометация сетевого оборудования	Взлом маршрутизаторов, коммутаторов, межсетевых экранов	Получение контроля над сетевой инфраструктурой для перехвата или блокировки трафика.
Клиентские угрозы	Социальная инженерия	Фишинг, Фишинг с подменой номера (vishing), Поддельные письма/звонки от техподдержки	Манипуляция пользователем с целью заставить его раскрыть конфиденциальные данные.
	Вредоносное ПО (Malware)	Кейлоггеры, Трояны (RAT), Шпионское ПО (Spyware), Рекламное ПО (Adware), Рансомверы	Установка на устройство пользователя вредоносного кода для кражи данных или контроля.
	Эксплуатация уязвимостей в ПО	Уязвимости в браузерах, Офисных пакетах (через вредоносные вложения), Медиаплеерах, PDF-ридерах	Использование ошибок в клиентских приложениях для выполнения произвольного кода.
	Атаки на веб-браузер	Drive-by-Download, Межсайтовый скриптинг (XSS), Подделка межсайтовых запросов (CSRF)	Загрузка и выполнение вредоносного кода при простом посещении скомпрометированного сайта.
	Кража данных и сессий	Кража файлов cookie сессии, Кража учетных данных, сохраненных в браузере	Несанкционированный доступ к активным сеансам и паролям пользователя.
Серверные угрозы	Внедрение кода	SQL-инъекции (SQLi), Инъекции команд ОС, Удаленное выполнение кода (RCE)	Отправка вредоносных данных на сервер, который интерпретирует их как код или команды.
	Некорректная аутентификация и авторизация	Небезопасное хранение паролей, Слабая политика паролей, Недостатки контроля доступа (Horizontal/Vertical Privilege Escalation)	Обход механизмов входа или получение прав, превышающих те, что предоставлены изначально.
	Раскрытие конфиденциальных	Небезопасные прямые ссылки на объекты (IDOR), Некорректные настройки	Непреднамеренный доступ к чужим данным из-за ошибок в логике

Уровень / Место возникновения	Категория угроз	Конкретные примеры	Краткое описание / Механизм
	данных	прав доступа к файлам и каталогам	приложения или настройках.
	Атаки на компоненты сервера	Эксплуатация уязвимостей в веб-сервере (Apache, Nginx), Сервере приложений, Системе управления контентом (CMS)	Использование уязвимостей в самом серверном ПО или платформах для получения контроля.
	Некорректная настройка безопасности	Открытые порты, Стандартные учетные записи и пароли, Отладочная информация, доступная извне	Атаки, возможные из-за ошибок администратора при развертывании и поддержке сервиса.

По типу воздействия

Категория воздействия	Нарушаемый принцип	Ключевые угрозы и атаки	Механизм воздействия
Раскрытие	Конфиденциальность (Confidentiality)	Перехват данных (Sniffing) Фишинг Социальная инженерия Кейлоггеры SQL-инъекции (кража данных) Слуфтинг (подслушивание)	Несанкционированное получение доступа к информации. Данные становятся известны тем, кому не должны быть доступны.
Изменение	Целостность (Integrity)	SQL-инъекции (изменение данных) Подмена данных (Data Spoofing) DNS-спуфинг Дефейс сайтов Манипуляция пакетами	Несанкционированное изменение или разрушение информации, что приводит к её недостоверности.
Уничтожение	Целостность (Integrity)	Рансомвер (шифрование данных) Стирание данных (wiper-вирусы) Повреждение системных файлов	Полное или частичное уничтожение информации, делающее её непригодной для использования.
Нарушение работы	Доступность (Availability)	DoS/DDoS-атаки Атаки на оборудование Рансомвер (блокировка доступа) Атаки на ресурсы системы	Создание условий, при которых легиментные пользователи не могут получить доступ к данным или системам.
Компрометация	Все три принципа	Внедрение backdoor Установка RAT (Remote Access Trojan) Создание ботнета	Полный захват контроля над системой, что позволяет нарушать все три принципа по желанию злоумышленника.

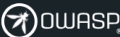
По принципу атаки

Принцип атаки	Категория угроз	Конкретные примеры	Механизм воздействия
Атаки на инфраструктуру	Сетевые атаки	DDoS, DNS-spoofing, ARP-poisoning, сканирование портов	Нацелены на сетевые компоненты, оборудование и каналы связи для нарушения доступности или перехвата трафика.

Принцип атаки	Категория угроз	Конкретные примеры	Механизм воздействия
	Атаки на оборудование	Физический доступ к серверам, Атаки на IoT-устройства, К/компрометация маршрутизаторов	Использование уязвимостей в физическом или сетевом оборудовании для получения контроля.
	Атаки на облачную инфраструктуру	Небезопасные конфигурации S3-бакетов, уязвимости оркестрации (Kubernetes)	Использование ошибок конфигурации облачных сервисов для доступа к данным или ресурсам.
Атаки на данные	Кража данных	SQL-инъекции, утечки данных через фишинг, эксплуатация misconfiguration (неправильных настроек) БД	Несанкционированное копирование, извлечение или просмотр конфиденциальной информации.
	Изменение данных	Подмена данных в БД, манипуляция финансовыми транзакциями, дефейс (изменение содержимого сайта)	Несанкционированная модификация данных для нанесения ущерба или мошенничества.
	Уничтожение данных	Ransomware, wiper-атаки, SQL-инъекции с оператором DROP	Удаление или шифрование данных с целью вывода систем из строя или вымогательства.
Атаки на код и логику приложения	Инъекционные атаки	SQLi, NoSQL-инъекции, Command Injection, LDAP-инъекции	Внедрение вредоносного кода/команд в интерпретируемую среду через некорректную обработку ввода.
	Ошибки обработки входных данных	Buffer Overflow, XSS (Cross-Site Scripting), XXE (XML External Entity)	Некорректная обработка пользовательского ввода, приводящая к выполнению кода или нарушению работы программы.
	Логические уязвимости	Небезопасная десериализация, IDOR (Insecure Direct Object References), уязвимости бизнес-логики	Использование ошибок в алгоритмах и workflow приложения для обхода защиты или получения неавторизованного доступа.
Атаки на механизмы аутентификации и авторизации	Обход аутентификации	Подбор учетных данных (Brute Force), использование утекших паролей, сессионный хикинг	Получение доступа к системе путем кражи, подбора или обхода проверки подлинности.
	Нарушение авторизации	Vertical/Horizontal Privilege Escalation, CSRF (Cross-Site Request Forgery)	Получение прав доступа, превышающих исходные, или выполнение действий от имени другого пользователя.
	Эксплуатация уязвимостей сессий	Фиксация сессии (Session Fixation), кража cookie	Перехват или угадывание идентификаторов сессии для несанкционированного доступа к учетной записи.

Ориентиром для понимания данных угроз был и остается [OWASP Top 10](#) — это стандартный отчет и регулярно обновляемый список из десяти самых критических и распространенных уязвимостей в безопасности веб-приложений.

Проект разрабатывается международным некоммерческим сообществом [OWASP \(Open Web Application Security Project\)](#), которое объединяет экспертов со всего мира. Этот документ служит главным ориентиром для разработчиков, тестировщиков и специалистов по информационной безопасности, помогая им проектировать защищенный софт и предотвращать потенциальные кибератаки.



[Home](#)
[Projects](#)
[Chapters](#)
[Events](#)
[Meetings](#)
[News](#)
[About](#)

[Sign in](#)
[Donate](#)

OWASP Top 10

The OWASP Top 10 is the reference standard for the most critical web application security risks. Adopting the OWASP Top 10 is perhaps the most effective first step towards changing your software development culture focused on producing secure code.

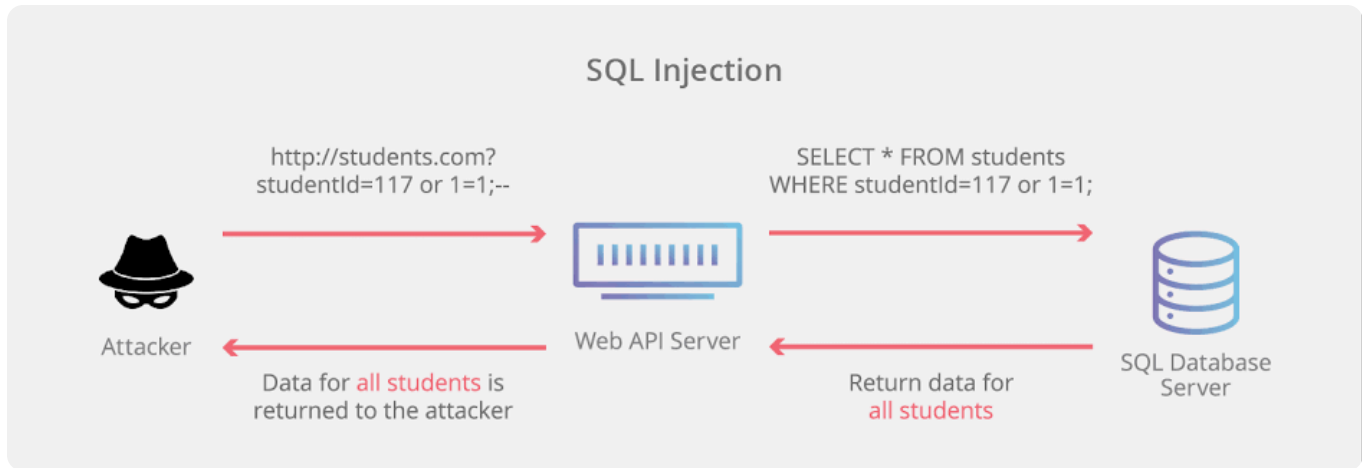
[View on GitHub](#)
[Documentation](#)



SQL Injection (SQLi)

SQL-инъекция (SQL injection, SQLi) — это один из самых распространенных и опасных способов взлома сайтов, при котором злоумышленник внедряет вредоносный код в запросы к базе данных.

Простыми словами, атака превращает обычное поле ввода на сайте (например, форму поиска или окно авторизации) в «командную строку» для базы данных. Если разработчик не защитил приложение, хакер может заставить сервер выполнить свои команды.



Рассмотреть данную угрозу можно на простом примере:

```
1 SELECT * FROM users WHERE username = '...' OR 1=1 --' AND password = '...'
```

SQL-инъекция в данном примере позволяет злоумышленнику обойти аутентификацию (войти в систему без знания логина и пароля), если введенные пользователем данные напрямую подставляются в строку SQL-запроса без фильтрации. Условие `1=1` всегда истинно (`TRUE`). Символы `--` (в зависимости от СУБД это может быть также `#` или `/*`) превращают всю оставшуюся часть запроса (включая проверку пароля) в комментарий. База данных проигнорирует проверку пароля и вернет первую запись из таблицы `users` (обычно это администратор).

Также данная строка может выглядеть так:

```
1 SELECT * FROM users WHERE username = '' OR '1'='1' AND password = '' OR '1'='1'
```

Если комментарии по какой-то причине не срабатывают, злоумышленник может внедрить условие, которое сделает истинным весь запрос целиком. Например, в поле `username` вводится `' OR '1'='1'`, а в поле `password` — `' OR '1'='1'`. Логическое выражение пересчитывается так, что итоговый результат проверки условий `WHERE` становится истинным. СУБД возвращает записи пользователей, и приложение успешно авторизует атакующего.

Способы защиты:

- **Параметризованный запрос** — это шаблон SQL-запроса, в котором данные отделены от кода и передаются как отдельные заполнители;
- **ORM (Object-Relational Mapping)** — это технология, которая связывает таблицы в базе данных с обычными классами и объектами в языках программирования (например, Python, Java или PHP); данная технология автоматически подставляет параметры в запросы (делает параметризацию);
- **Экранирование** — это метод защиты объектов, данных или оборудования от внешних нежелательных воздействий с помощью специального барьера (экрана);
- **Принцип наименьших привилегий (Principle of Least Privilege, PoLP)** — это концепция безопасности, которая означает предоставление пользователям, программам или системам только минимально необходимых прав и доступов для выполнения их текущих задач.

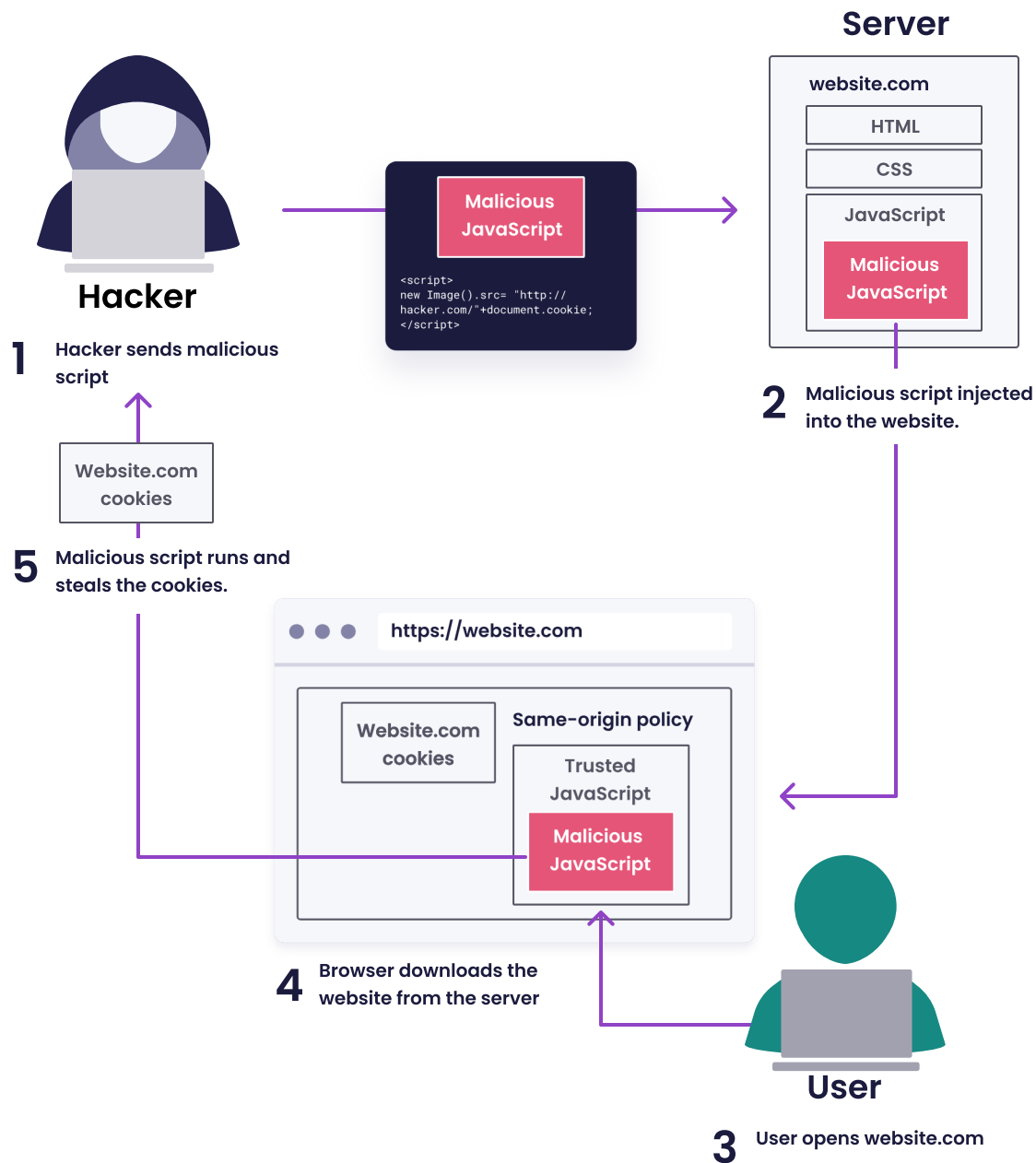
Cross-Site Scripting (XSS)

XSS (Cross-Site Scripting, или межсайтовый скриптинг) — это одна из самых распространенных уязвимостей в веб-приложениях, при которой злоумышленник внедряет вредоносный JavaScript-код в страницу, отображаемую в браузере обычного пользователя.

XSS возникает, когда веб-приложение принимает данные от пользователя (например, комментарий, поисковый запрос или параметр в URL) и выводит их на страницу без предварительной очистки и экранирования. Вместо обычного текста (например, имени) злоумышленник может отправить конструкцию вида `<script>вредный_код(</script>` . Браузер жертвы, встретив этот тег, сразу запустит написанный сценарий.

Существуют основные виды, такие как:

- **Хранимые (Stored/Persistent XSS)** — самый опасный тип. Скрипт навсегда сохраняется в базе данных сервера (например, в тексте статьи или комментарии на форуме). Каждый раз, когда любой пользователь открывает эту страницу, вредоносный код автоматически срабатывает в его браузере;
- **Отраженные (Reflected/Non-persistent XSS)** — скрипт не сохраняется на сервере, а передается «на лету» в составе вредоносной ссылки. Атака сработает только в том случае, если жертва сама кликнет по такой специальной ссылке;
- **DOM-based XSS** — атака происходит исключительно на стороне клиента (в браузере) без участия серверного кода. Вредоносные данные обрабатываются некорректно написанными скриптами JavaScript самого сайта, которые динамически меняют структуру страницы (DOM-дерево).



Пример уязвимого кода:

```
1 function handleMessageSend(messageId, senderEmail, messageContent) {
2     database.save(messageId, senderEmail, messageContent);
3 }
4
5 function generateMessageHTML(messageId) {
6     let messageContent = database.loadContent(messageId);
7     return `
```

Код берет текст сообщения messageContent (вторая функция) напрямую из базы данных и вставляет его в HTML-разметку без какой-либо очистки или фильтрации. Если злоумышленник отправит сообщение, содержащее вредоносный JavaScript-код, то этот код сохранится в базу. Затем, когда любой другой пользователь откроет страницу с этим сообщением, его браузер выполнит этот скрипт. Это позволит хакеру украсть сессии, куки или выполнить действия от имени жертвы.

Способы защиты:

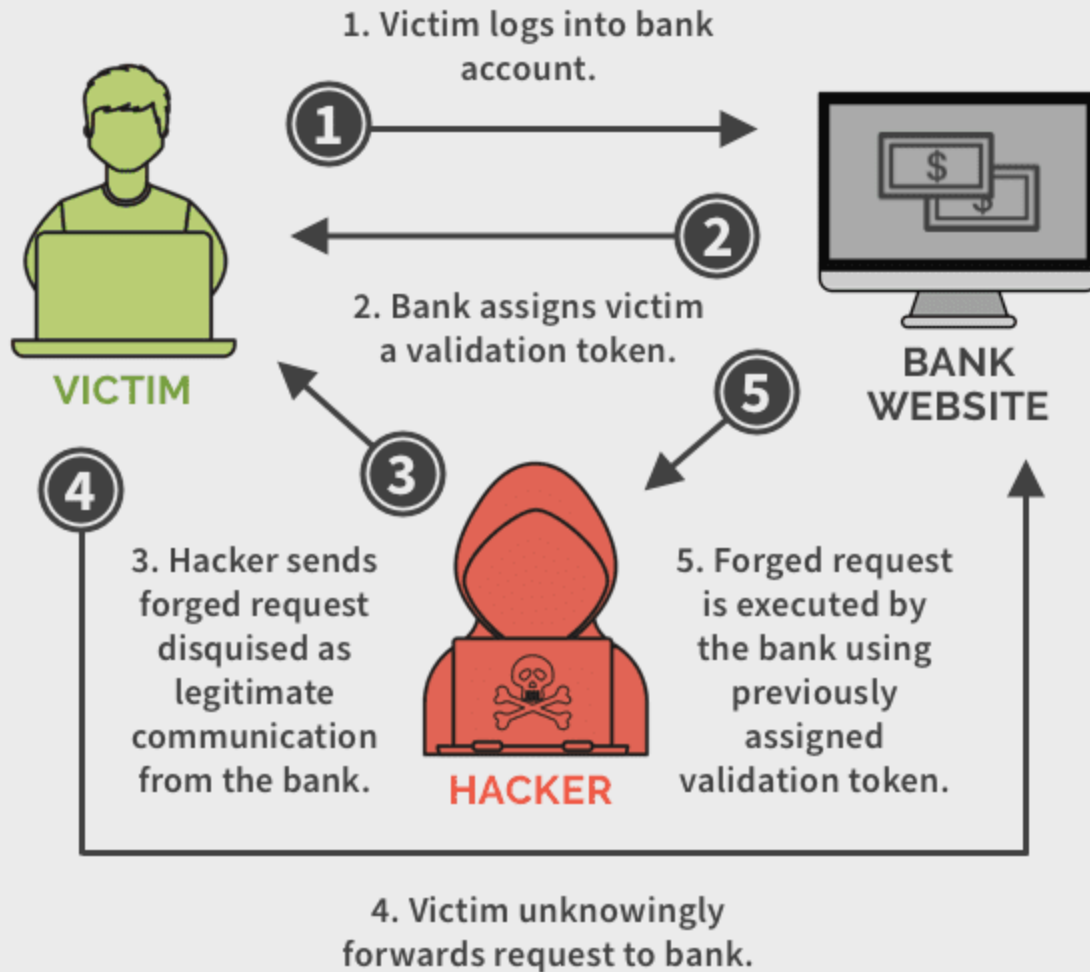
- **Экранирование HTML-сущностей** — это преобразование специальных символов (`<` , `>` , `"` , `'` , `&`) в безопасные HTML-коды (`<` , `>` , `"` , `'` , `&`) перед выводом данных на страницу, чтобы браузер отображал их как текст, а не выполнял как код;
- **Content Security Policy (CSP)** — это HTTP-заголовок, который указывает браузеру, из каких источников разрешено загружать и выполнять скрипты, стили и другие ресурсы, блокируя внедрённый вредоносный JavaScript;
- **Флаг HttpOnly для cookie** — это атрибут cookie, который запрещает доступ к ним через JavaScript, предотвращая кражу сессионных данных при успешной XSS-атаке;
- **Контекстное экранирование** — это выбор способа кодирования данных в зависимости от места вывода (HTML, JavaScript, URL, CSS), чтобы данные всегда интерпретировались как текст, а не как исполняемый код;
- **Санитизация ввода** — это очистка пользовательских данных от потенциально опасных конструкций (тегов `<script>` , обработчиков событий и т.п.) перед сохранением или отображением.

Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF / XSRF) — это вид кибератаки на веб-системы, при которой злоумышленник заставляет браузер жертвы выполнить нежелательное действие на доверенном сайте, где пользователь в данный момент авторизован. На русском языке эта уязвимость называется межсайтовой подделкой запроса.

Главная особенность CSRF заключается в том, что вредоносный сайт использует активную сессию пользователя (его файлы cookie). Браузер автоматически прикрепляет сессионные куки к любому запросу к целевому сайту, даже если этот запрос был инициирован со сторонней страницы. Сама по себе атака CSRF не позволяет злоумышленнику украсть данные пользователя или прочитать ответ сервера (этому мешает правило ограничения домена SOP), но она позволяет совершать деструктивные действия от его имени.

Cross-Site Request Forgery



Способы защиты:

- **CSRF-токен** — это уникальный, непредсказуемый секретный параметр, который сервер генерирует для каждой формы или сессии, вставляет в скрытое поле и проверяет при получении запроса, подтверждая, что запрос отправлен именно с легитимной страницы сайта;
- **Атрибут SameSite для cookie** — это параметр cookie (Strict или Lax), который запрещает браузеру отправлять cookie вместе с межсайтовыми запросами, тем самым блокируя автоматическую передачу сессии на вредоносный сайт;
- **Проверка заголовков Origin и Referer** — это серверная проверка источника запроса: сервер сравнивает значение заголовков Origin или Referer с ожидаемым доменом и отклоняет запрос, если он пришёл с постороннего сайта;
- **Повторная аутентификация для критичных операций** — это требование ввести пароль, код из SMS или подтвердить действие заново перед выполнением важных действий (перевод денег, смена пароля и т.п.), даже если пользователь уже авторизован;
- **Метод Double Submit Cookie** — это техника, при которой токен одновременно хранится в cookie и передаётся в теле запроса (или заголовке), а сервер проверяет их совпадение, не требуя хранения токена на стороне сервера.

Безопасная обработка данных и сопутствующие практики

Валидация — это комплексный процесс проверки того, что защитные меры, системы или входящие данные реально выполняют свои функции и обеспечивают необходимый уровень безопасности в условиях актуальных угроз.

Санитизация — это процесс изменения или удаления опасных и некорректных элементов из входящих данных, чтобы сделать их абсолютно безопасными для дальнейшей обработки системой.

Экранирование — это технический прием, который превращает управляющие символы кода в безопасный текст. Он заставляет систему (браузер, базу данных, операционную систему) воспринимать пользовательский ввод исключительно как строку данных, а не как команду для исполнения.

Главное правило работы с данными звучит так: **валидируй на входе, экранируй на выходе**. Эти механизмы не заменяют, а дополняют друг друга.

Валидация, санитизация и экранирование: в чём разница

Критерий	Валидация	Санитизация	Экранирование
Что делает	Проверяет, соответствуют ли данные ожидаемому формату, типу, длине и диапазону.	Удаляет или изменяет опасные элементы, сохраняя остальное содержимое.	Превращает спецсимволы в безопасное представление, чтобы они воспринимались как текст.
Когда применяется	На входе: сразу при получении данных.	На входе или перед выводом, когда нужно разрешить часть разметки.	На выходе: в момент подстановки данных в HTML, SQL, URL или команду ОС.
Судьба данных	Принимаются или отклоняются целиком.	Изменяются (опасная часть удаляется).	Смысл не меняется, меняется только форма представления.
Пример	Логин: только латиница, цифры и <code>_</code> , длина 3–20 символов; email соответствует формату.	Комментарий с форматированием: теги <code></code> , <code><i></code> остаются, <code><script></code> удаляется.	<code><script></code> превращается в <code>&lt;script&gt;</code> .
Типичный инструмент	Регулярные выражения, схемы данных (JSON Schema, Zod), встроенные валидаторы фреймворков.	Специализированные библиотеки (например, DOMPurify).	Шаблонизаторы с автоэкранированием, параметризованные запросы.

Когда что применять:

- Валидация** — всегда и обязательно на стороне сервера, по принципу **белого списка** (разрешено только то, что явно описано). «Чёрные списки» запрещённых символов легко обойти. Проверка на стороне клиента нужна только для удобства пользователя;
- Санитизация** — когда нужно сохранить часть пользовательской разметки (текстовый редактор, комментарии с форматированием). Использовать следует проверенную библиотеку, а не самописные регулярные выражения;
- Экранирование** — всегда при выводе данных, причём способ экранирования зависит от контекста (HTML, JavaScript, URL, SQL).

Важно понимать, что валидация не отменяет экранирование. Например, фамилия `O'Brien` абсолютно корректна и пройдёт любую валидацию, но апостроф в ней опасен для SQL-запроса. Поэтому проверка формата и безопасная подстановка (параметризованный запрос) решают разные задачи.

```
1 // 1. Валидация (белый список)
2 const USERNAME_REGEX = /^[a-zA-Z0-9_]{3,20}$/;
3
4 if (!USERNAME_REGEX.test(username)) {
5   throw new Error('Недопустимое имя пользователя');
6 }
7
8 // 2. Санитизация (разрешённая разметка остаётся, опасная удаляется)
9
10 const cleanHtml = DOMPurify.sanitize(userComment);
11
12 // 3. Экранирование при выводе (браузер воспримет данные как текст, а не как код)
13 element.textContent = userComment;
```

Безопасное хранение паролей

Пароли нельзя хранить в открытом виде: при утечке базы данных злоумышленник сразу получит доступ ко всем аккаунтам. Шифрование тоже не подходит, так как оно обратимо, а ключ может быть украден вместе с базой. Правильное решение — хеширование.

Хеширование — это одностороннее преобразование данных (пароля) в строку фиксированной длины, которое невозможно обратить. Для проверки пароля приложение хеширует введённое значение и сравнивает его с сохранённым хешем.

Однако обычные алгоритмы (MD5, SHA-1, SHA-256) создавались для скорости, и современные видеокарты способны перебирать миллиарды вариантов в секунду. Поэтому для паролей используются **специальные медленные алгоритмы**, которые намеренно требуют много времени и памяти.

Алгоритм	Статус	Особенности
Argon2id	Рекомендуется для новых систем	Требует много памяти и времени (memory-hard), устойчив к перебору на GPU и специализированных чипах (ASIC). Описан в стандарте RFC 9106.
bcrypt	Допустим, особенно в уже существующих системах	Настраиваемый «фактор работы» (рекомендуется 10 и выше). Ограничение: учитывает только первые 72 байта пароля.
PBKDF2	Допустим (например, при требованиях FIPS)	Многократное применение HMAC. Менее устойчив к атакам на GPU, чем Argon2id.
MD5, SHA-1, SHA-256	Не использовать для паролей	Слишком быстрые, позволяют быстро перебирать пароли. SHA-256 хорош для контроля целостности, но не для хранения паролей.

Соль (salt) — это уникальная случайная строка, которая добавляется к паролю перед хешированием. Она нужна для того, чтобы:

- одинаковые пароли разных пользователей давали разные хеши;
- радужные таблицы (заранее вычисленные хеши популярных паролей) стали бесполезны;
- злоумышленник не мог взламывать пароли всех пользователей одновременно.

Соль не является секретом и хранится рядом с хешем. Современные библиотеки (Argon2, bcrypt) генерируют соль автоматически и записывают её прямо в итоговую строку:

```
1 $argon2id$v=19$m=19456,t=2,p=1$<соль>$<хеш>
```

Здесь **argon2id** — алгоритм, **v=19** — версия, **m** — объём памяти в КиБ (19456 КиБ ≈ 19 МиБ), **t** — число итераций, **p** — степень параллелизма.

Рекомендации:

- использовать проверенные библиотеки, а не писать собственную реализацию хеширования;
- подбирать параметры алгоритма так, чтобы одно хеширование занимало заметную долю секунды на сервере (это почти не влияет на пользователя, нократно замедляет перебор);
- не ограничивать длину пароля слишком жёстко и разрешать любые символы, поощряя длинные парольные фразы;
- проверять новые пароли по базам утёкших паролей;
- сравнивать хеши только средствами самой библиотеки.

Защита сессий

HTTP не хранит состояние между запросами, поэтому после входа сервер выдаёт пользователю **идентификатор сессии (Session ID)** — случайную строку, которая хранится в cookie и отправляется с каждым запросом. Тот, кто владеет этим идентификатором, для сервера «является» пользователем, поэтому Session ID — главная цель атак (кража через XSS, перехват в сети, фиксация сессии).

Атрибуты cookie для защиты сессии:

- **HttpOnly** — это атрибут cookie, который запрещает доступ к ней через JavaScript (`document.cookie`), поэтому даже при успешной XSS-атаке злоумышленник не сможет прочесть идентификатор сессии;
- **Secure** — это атрибут cookie, который разрешает браузеру отправлять её только по защищённому соединению HTTPS, исключая передачу идентификатора по открытому каналу;
- **SameSite** — это атрибут cookie, который ограничивает отправку её при межсайтовых запросах и защищает от CSRF (подробнее о значениях ниже);
- **Префикс** — это дополнительная защита в имени cookie: браузер примет такую cookie, только если у неё установлен **Secure** , путь `/` и не указан **Domain** , что не даёт подменить её с поддомена.

Значение SameSite	Поведение
Strict	Cookie не отправляется ни при каких межсайтовых запросах, даже при переходе по ссылке с другого сайта. Максимальная защита, но менее удобно.
Lax	Cookie отправляется при переходе по ссылке (обычный GET-запрос), но не отправляется при межсайтовых POST-запросах, в картинках и фреймах. Разумный компромисс.
None	Cookie отправляется всегда. Требует атрибута <code>Secure</code> . Используется только когда межсайтовая отправка действительно необходима.

Пример безопасно настроенной cookie сессии:

```
1 Set-Cookie: __Host-sessionid=a3fWa9x...; Secure; HttpOnly; SameSite=Lax; Path=/
```

Регенерация Session ID после входа

Фиксация сессии (Session Fixation) — это атака, при которой злоумышленник заставляет жертву использовать заранее известный ему идентификатор сессии. Атака проходит в несколько шагов:

1. Злоумышленник получает от сайта действительный идентификатор сессии (пока без авторизации);
2. Он подсовывает этот идентификатор жертве (например, через специально сформированную ссылку);
3. Жертва входит в свой аккаунт, и если идентификатор при этом не меняется, сессия с этим ID становится авторизованной;
4. Злоумышленник использует тот же ID и попадает в аккаунт без знания пароля.

Защита проста: **после успешного входа необходимо создать новый Session ID, а старый уничтожить**. То же правило действует при смене прав доступа и пароля.

```
1 app.post('/login', (req, res) => {
2   const user = authenticate(req.body.username, req.body.password);
3   if (!user) return res.status(401).send('Неверные данные');
4
5   req.session.regenerate((err) => {           // создаётся новый Session ID
6     if (err) return res.status(500).send('Ошибка сервера');
7     req.session.userId = user.id;
8     res.redirect('/profile');
9   });
10 });
```

Дополнительные меры:

- генерировать идентификаторы криптографически стойким генератором случайных чисел и достаточной длины, чтобы их нельзя было угадать;
- ограничивать время жизни сессии (по бездействию и абсолютное);
- при выходе из системы уничтожать сессию на сервере, а не только удалять cookie в браузере;
- использовать механизм сессий проверенного фреймворка, а не писать собственный.

Безопасные HTTP-заголовки

HTTP-заголовки безопасности — это инструкции, которые сервер передаёт браузеру вместе с ответом и которые включают встроенные защитные механизмы браузера. Они почти ничего не стоят в реализации, но закрывают целые классы атак.

Основные заголовки:

- **Content-Security-Policy (CSP)** — это заголовок, который задаёт список источников, откуда браузеру разрешено загружать и выполнять скрипты, стили, изображения и другие ресурсы. Даже если злоумышленнику удалось внедрить `<script>` в страницу, браузер откажется его выполнять (защита от XSS);
- **X-Content-Type-Options: nosniff** — это заголовок, который запрещает браузеру «угадывать» тип файла по его содержимому и заставляет доверять только заявленному типу (`Content-Type`). Он защищает, например, от ситуации, когда загруженный пользователем файл выдаёт себя за картинку, а браузер выполняет его как скрипт;
- **Strict-Transport-Security (HSTS)** — это заголовок, который требует от браузера обращаться к сайту только по HTTPS в течение заданного срока, даже если пользователь ввёл адрес с `http://` . Он защищает от атаки «понижения протокола» и перехвата трафика (MitM).

```

1 Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'
2 X-Content-Type-Options: nosniff
3 Strict-Transport-Security: max-age=31536000; includeSubDomains

```

Разбор примера:

- `default-src 'self'` — по умолчанию ресурсы разрешено загружать только с собственного домена;
- `script-src 'self'` — скрипты разрешены только с собственного домена, а встроенные (inline) скрипты блокируются;
- `object-src 'none'` — запрет плагинов и встроенных объектов;
- `base-uri 'self'` — запрет подмены базового адреса страницы;
- `max-age=31536000` — срок действия правила HSTS в секундах (один год);
- `includeSubDomains` — правило распространяется на все поддомены.

Принцип наименьших привилегий и минимизация поверхности атаки

Принцип наименьших привилегий (PoLP) уже упоминался среди базовых принципов и в защите от SQL-инъекций. Его смысл в том, что даже при успешной атаке ущерб должен быть ограничен правами скомпрометированного компонента.

Уровень	Нарушение принципа	Правильный подход
База данных	Приложение подключается к СУБД под администратором (root).	Отдельная учётная запись с правами только на нужные таблицы и операции.
Операционная система	Веб-сервер или сервис запущен от имени root.	Запуск от отдельного непривилегированного пользователя.
Приложение	Обычный пользователь может вызвать административные функции.	Ролевая модель доступа (RBAC), проверка прав на сервере при каждом запросе.
Файлы и секреты	Ключи и конфигурации доступны всем процессам и пользователям.	Доступ только тем, кому он необходим для работы.

```

1 -- Приложению нужны только чтение и запись в таблицу заказов, и ничего более
2 CREATE USER app_user WITH PASSWORD '*****';
3 GRANT SELECT, INSERT, UPDATE ON orders TO app_user;

```

Если в приложении окажется SQL-инъекция, то при таких правах злоумышленник не сможет удалить таблицы или прочитать данные из других частей системы.

Минимизация поверхности атаки — это сокращение числа точек, через которые систему можно атаковать: чем меньше кода, сервисов и открытых входов, тем меньше потенциальных уязвимостей. На практике это означает:

- отключать неиспользуемые сервисы, порты и модули;
- удалять отладочные страницы, тестовые эндпоинты и учётные записи по умолчанию;
- убирать неиспользуемые библиотеки и зависимости;
- закрывать админ-панели от публичного доступа (по IP-адресам или через VPN);
- не раскрывать лишнее в сообщениях об ошибках и заголовках ответа (версии ПО, трассировки стека).

Инструменты разработчика

Безопасность не должна проверяться только перед релизом: инструменты анализа встраивают в процесс разработки (например, в автоматическую сборку и тестирование, CI/CD), чтобы уязвимости находились как можно раньше и обходились дешевле.

SAST (Static Application Security Testing) — это статический анализ безопасности: автоматическая проверка исходного кода без запуска программы. Инструмент ищет опасные конструкции (склеивание SQL-запросов из пользовательских данных, вывод без экранирования, пароли, прописанные прямо в коде). К плюсам относится раннее обнаружение ошибок, а к минусам — ложные срабатывания и невозможность увидеть проблемы настройки окружения. Примеры инструментов: Semgrep, SonarQube.

SCA (Software Composition Analysis) — это анализ состава программного обеспечения: проверка сторонних библиотек и зависимостей на наличие известных уязвимостей. Он необходим, потому что значительная часть современного приложения — это чужой код (пакеты npm, pip, Maven), и уязвимость в одной библиотеке делает уязвимым всё приложение.

Основные инструменты SCA:

- **OWASP Dependency-Check** — это бесплатный инструмент, который определяет зависимости проекта, сопоставляет их с идентификаторами CPE и формирует отчёт со списком связанных уязвимостей (CVE). Данные берутся из базы NVD и обновляются автоматически; инструмент можно подключить к Maven, Gradle, Jenkins и GitHub Actions;
- **npm audit** — это встроенная команда менеджера пакетов npm, которая проверяет зависимости Node.js-проекта на известные уязвимости;
- **Snyk** — это коммерческая платформа (с бесплатным тарифом), которая ищет уязвимости в зависимостях и предлагает варианты исправления.

```
1 npm audit # проверка зависимостей проекта на известные уязвимости
2 npm audit fix # автоматическое обновление до безопасных версий
```

Мониторинг CVE

CVE (Common Vulnerabilities and Exposures) — это уникальный идентификатор публично известной уязвимости в формате **CVE-ГОД-НОМЕР** (например, **CVE-2021-44228** — знаменитая уязвимость Log4Shell в библиотеке Log4j). Для оценки серьёзности используется шкала **CVSS** от 0 до 10, а подробные описания собраны в **NVD (National Vulnerability Database)** — национальной базе уязвимостей США.

Мониторинг должен быть **регулярным**, а не разовым: новые уязвимости публикуются ежедневно, и библиотека, безопасная вчера, сегодня может оказаться уязвимой. На практике это означает:

- автоматически запускать SCA-проверку при каждой сборке проекта;
- подписываться на уведомления об уязвимостях в используемых технологиях;
- в первую очередь исправлять уязвимости с высоким баллом CVSS и те, которые уже эксплуатируются в реальных атаках (для этого существует каталог CISA KEV);
- своевременно обновлять зависимости и убирать ненужные.

Инструмент	Тип	Что проверяет	Пример находки
Semgrep, SonarQube	SAST	Собственный исходный код	SQL-запрос, собранный конкатенацией строк
OWASP Dependency-Check, npm audit, Snyk	SCA	Сторонние библиотеки и зависимости	Устаревшая версия библиотеки с известным CVE
NVD, CVE, CISA KEV	Базы данных	Информация об уязвимостях	Описание, оценка CVSS, признак активной эксплуатации